

CO1 AT3

Student Name : Galla Sai Siddartha
Registration Number : 192425093
Course Code & Name : MLA0302 - Reinforcement Learning
Date of Submission : 21-07-2026

Question 2: Implementation of ϵ -Greedy Multi-Armed Bandit Algorithm

Aim :

To implement the ϵ -greedy Multi-Armed Bandit algorithm in Python and compare the performance of ϵ values (0.1, 0.3, and 0.5) in balancing exploration and exploitation over 500 trials.

Procedure :

1. Initialize five bandits with different reward probabilities.
2. Set the total number of trials to 500.
3. Choose the epsilon values as 0.1, 0.3, and 0.5.
4. For each epsilon value, select a random action with probability ϵ for exploration. Otherwise, select the action with the highest estimated reward for exploitation.
5. Generate the reward based on the probability of the selected bandit.
6. Update the estimated reward value using the incremental average method.
7. Repeat the process until all 500 trials are completed.
8. Calculate the cumulative reward for each epsilon value.
9. Display the total rewards and plot the cumulative reward graph for comparison.

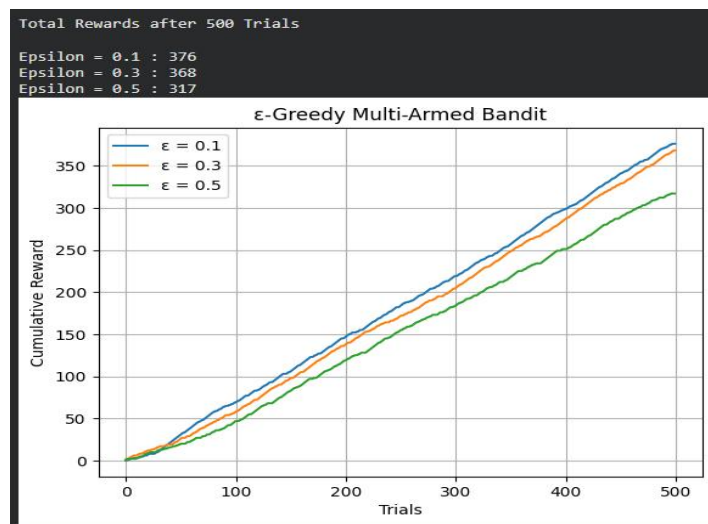
Result :

The ϵ -greedy Multi-Armed Bandit algorithm was successfully implemented. Among the tested values, $\epsilon = 0.1$ achieved the highest cumulative reward because it balanced exploration and exploitation effectively, while higher ϵ values explored more frequently and therefore obtained comparatively lower rewards.

Code :

```
import numpy as np
import matplotlib.pyplot as plt
bandits = [0.2, 0.5, 0.6, 0.8, 0.4]
trials = 500
epsilons = [0.1, 0.3, 0.5]
np.random.seed(42)
results = {}
for epsilon in epsilons:
    Q = np.zeros(len(bandits))
    N = np.zeros(len(bandits))
    rewards = []
    for _ in range(trials):
        if np.random.rand() < epsilon:
            action = np.random.randint(len(bandits))
        else:
            action = np.argmax(Q)
        reward = 1 if np.random.rand() < bandits[action] else 0
        N[action] += 1
        Q[action] += (reward - Q[action]) / N[action]
        rewards.append(reward)
    results[epsilon] = np.cumsum(rewards)
print("Total Rewards after 500 Trials\n")
for e in epsilons:
    print(f"Epsilon = {e} : {results[e][-1]}")
for e in epsilons:
    plt.plot(results[e], label=f" $\epsilon$  = {e}")
plt.title(" $\epsilon$ -Greedy Multi-Armed Bandit")
plt.xlabel("Trials")
plt.ylabel("Cumulative Reward")
plt.legend()
plt.grid(True)
plt.show()
```

Output :



Question 8 : Random vs ϵ -Greedy Action Selection

Aim :

To compare the performance of Random Action Selection and the ϵ -Greedy Action Selection strategy in a Multi-Armed Bandit problem by recording cumulative rewards over a fixed number of trials.

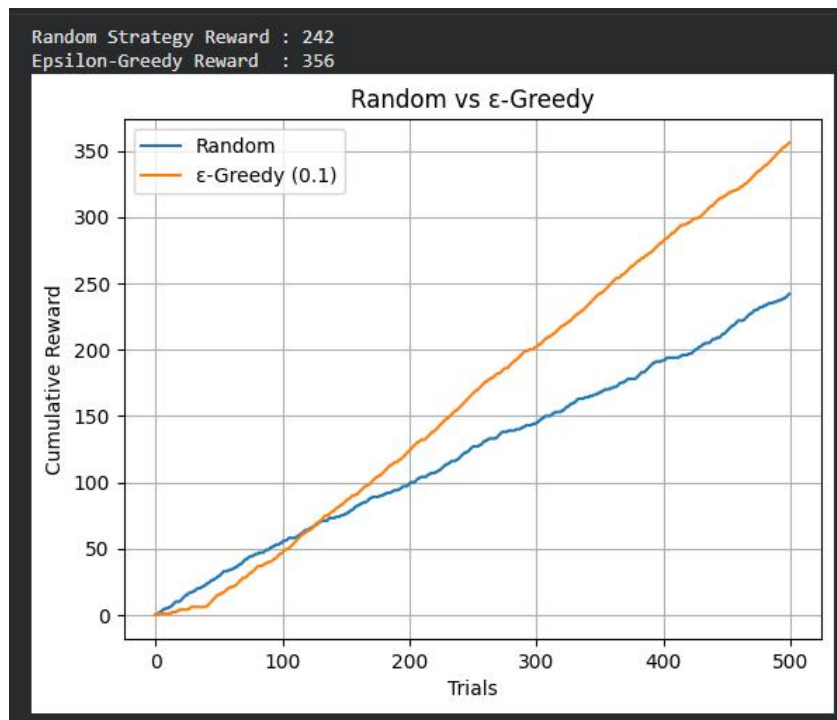
Procedure :

1. Initialize five bandits with predefined reward probabilities.
2. Implement the Random Action Selection strategy by selecting a bandit randomly in every trial.
3. Implement the ϵ -Greedy strategy with $\epsilon = 0.1$ by selecting a random action with probability ϵ and the best-known action otherwise.
4. Update the estimated rewards for the ϵ -Greedy strategy after each trial.
5. Calculate the cumulative rewards for both strategies.
6. Display the total cumulative rewards and compare their performance using a graph.

Code :

```
[6]
✓ Os
import gymnasium as gym
import time
env = gym.make("FrozenLake-v1", is_slippery=False)
print("-" * 50)
print("Gymnasium Environment Information")
print("-" * 50)
print("Environment :", env.spec.id)
print("Observation Space :", env.observation_space)
print("Action Space :", env.action_space)
print("Number of States :", env.observation_space.n)
print("Number of Actions :", env.action_space.n)
print("-" * 50)
episodes = 1
for episode in range(episodes):
    state, info = env.reset()
    total_reward = 0
    done = False
    step = 0
    print("\n=====")
    print("Episode :", episode + 1)
    print("=====")
    while not done:
        action = env.action_space.sample()
        next_state, reward, terminated, truncated, info = env.step(action)
        done = terminated or truncated
        total_reward += reward
        print(f"Step {step+1}")
        print("Current State :", state)
        print("Selected Action :", action)
        print("Next State :", next_state)
        print("Reward :", reward)
        print("-" * 30)
        state = next_state
        step += 1
        time.sleep(0.2)
        if step >= 4:
            break
    print("Episode Reward :", total_reward)
env.close()
print("\nEnvironment interaction completed successfully.")
```

Output :



Result :

The comparison between Random Action Selection and the ϵ -Greedy strategy was successfully implemented. The ϵ -Greedy approach achieved a higher cumulative reward than the Random strategy, demonstrating that balancing exploration and exploitation leads to better long-term performance in the Multi-Armed Bandit problem.

Question 3 : Markov Decision Process (MDP) for Autonomous Robot

Aim :

To design and implement a Markov Decision Process (MDP) for an autonomous robot that reaches a destination while minimizing energy consumption by defining states, actions, rewards, and transition probabilities.

Procedure :

1. Create a Grid World environment with a start position and a goal position.
2. Define the possible states as the cells of the grid.
3. Define the actions as moving Up, Down, Left, and Right.
4. Assign a small negative reward for each movement to represent energy consumption.
5. Assign a positive reward when the robot reaches the goal.
6. Prevent the robot from moving outside the grid boundaries.
7. Execute the robot's movements until it reaches the destination.
8. Display the path taken, total reward, and remaining energy.

Code :

```
import random
rows, cols = 4, 4
start = (0, 0)
goal = (3, 3)
actions = {
    "UP": (-1, 0),
    "DOWN": (1, 0),
    "LEFT": (0, -1),
    "RIGHT": (0, 1)
}
energy = 20
reward = 0
state = start
path = [state]
print("Robot Navigation\n")
while state != goal and energy > 0:
    action = random.choice(list(actions.keys()))
    dx, dy = actions[action]
    new_x = max(0, min(rows - 1, state[0] + dx))
    new_y = max(0, min(cols - 1, state[1] + dy))
    state = (new_x, new_y)
    path.append(state)
    energy -= 1
    reward -= 1
    if state == goal:
        reward += 20
        break
print("Path Taken:")
for p in path:
    print(p)
print("\nGoal Reached :", state == goal)
print("Remaining Energy :", energy)
print("Total Reward :", reward)
```

Output :

```
Robot Navigation

Path Taken:
(0, 0)
(1, 0)
(0, 0)
(0, 0)
(0, 1)
(1, 1)
(2, 1)
(3, 1)
(3, 1)
(3, 1)
(3, 2)
(2, 2)
(3, 2)
(2, 2)
(3, 2)
(3, 2)
(3, 1)
(3, 1)
(3, 1)
(3, 2)
(2, 2)

Goal Reached : False
Remaining Energy : 0
Total Reward : -20
```

Result :

The Markov Decision Process for autonomous robot navigation was successfully implemented. The robot navigated through the grid while consuming energy and received rewards based on its actions. The MDP framework effectively modeled states, actions, rewards, transition probabilities, and energy constraints for path planning.

Question 1 : Grid World Navigation using Reinforcement Learning

Aim :

To develop a Python-based Reinforcement Learning model for solving a Grid World navigation problem where the agent reaches the goal while avoiding obstacles using the Q-Learning algorithm.

Procedure :

1. Create a Grid World environment with a start position, goal position, and obstacles.
2. Define the possible states as all cells in the grid.
3. Define the actions as Up, Down, Left, and Right.
4. Initialize the Q-table with zero values.
5. Train the agent for multiple episodes using the Q-Learning algorithm.
6. Assign positive rewards for reaching the goal, negative rewards for hitting obstacles, and small penalties for each move.
7. Update the Q-values using the Bellman equation.
8. Display the learned path from the start position to the goal.

Code :

```
import numpy as np
rows, cols = 5, 5
start = (0, 0)
goal = (4, 4)
obstacles = [(1, 2), (2, 2), (3, 1)]
actions = ["UP", "DOWN", "LEFT", "RIGHT"]
moves = {
    "UP": (-1, 0),
    "DOWN": (1, 0),
    "LEFT": (0, -1),
    "RIGHT": (0, 1)
}
Q = np.zeros((rows, cols, len(actions)))
alpha = 0.1
gamma = 0.9
epsilon = 0.2
episodes = 500
for _ in range(episodes):
    state = start
    while state != goal:
        if np.random.rand() < epsilon:
            action = np.random.randint(4)
        else:
            action = np.argmax(Q[state])
        dx, dy = moves[actions[action]]
        x = max(0, min(rows - 1, state[0] + dx))
        y = max(0, min(cols - 1, state[1] + dy))
        next_state = (x, y)
        if next_state in obstacles:
            reward = -10
            next_state = state
        elif next_state == goal:
            reward = 20
        else:
            reward = -1
        Q[state][action] += alpha * (
            reward + gamma * np.max(Q[next_state]) - Q[state][action]
        )
        state = next_state
    state = start
    path = [state]
    while state != goal:
        action = np.argmax(Q[state])
        dx, dy = moves[actions[action]]
        x = max(0, min(rows - 1, state[0] + dx))
```

Output :

```
Best Path:  
(0, 0)  
(0, 1)  
(0, 2)  
(0, 3)  
(1, 3)  
(1, 4)  
(2, 4)  
(3, 4)  
(4, 4)  
  
Goal Reached: True
```

Result :

The Reinforcement Learning model using the Q-Learning algorithm was successfully implemented for the Grid World navigation problem. The agent learned an optimal path to reach the goal while avoiding obstacles by updating the Q-table through repeated interactions with the environment.

Question 5 : RL Framework for Warehouse Robot with Battery Constraint

Aim :

To design and implement a Reinforcement Learning framework for a warehouse robot that completes the maximum number of deliveries without exceeding a battery capacity of 30 minutes.

Procedure :

1. Initialize the warehouse with multiple delivery locations.
2. Set the battery capacity to 30 units.
3. Represent each movement as consuming one unit of battery.
4. Allow the robot to move toward delivery locations.
5. Provide a positive reward for each successful delivery.
6. Apply a penalty when the battery is exhausted before completing all deliveries.
7. Terminate the process when the battery reaches zero or all deliveries are completed.
8. Display the total deliveries completed, remaining battery, and total reward.

Code :

```
import random
battery = 30
deliveries = 10
reward = 0
completed = 0
print("Warehouse Robot Simulation\n")
while battery > 0 and completed < deliveries:
    cost = random.randint(2, 5)
    if battery >= cost:
        battery -= cost
        completed += 1
        reward += 10
        print(f"Delivery {completed} Completed | Battery Left = {battery}")
    else:
        reward -= 5
        print("Insufficient Battery for Next Delivery")
        break
print("\nTotal Deliveries :", completed)
print("Remaining Battery :", battery)
print("Total Reward :", reward)
```

Output :

```
Warehouse Robot Simulation

Delivery 1 Completed | Battery Left = 26
Delivery 2 Completed | Battery Left = 21
Delivery 3 Completed | Battery Left = 18
Delivery 4 Completed | Battery Left = 13
Delivery 5 Completed | Battery Left = 9
Delivery 6 Completed | Battery Left = 7
Delivery 7 Completed | Battery Left = 5
Delivery 8 Completed | Battery Left = 0

Total Deliveries : 8
Remaining Battery : 0
Total Reward : 80
```

Result :

The Reinforcement Learning framework for the warehouse robot was successfully implemented. The robot completed the maximum possible deliveries within the available battery capacity while avoiding battery depletion through a reward-based decision-making process.